Lab 3: Monitoring, Logging & DevSecOps

Task 1 — Implement Centralized Logging

Log collection, Grok parsing pipeline, Elasticsearch indexing, and Kibana visualization

Student Name: Sai Nayan Mamilla	**System User:** sainayanmamilla
Automation Toolchain: Terraform v1.16 & Ansible v2.21	**Host Machine:** Sais-MacBook-Pro.local (macOS arm64)
Assessment Platform: Zosima.ai Lab 3	**Deliverable:** Task 1 Submission (5 Marks)

1. Experiment Objectives & System Architecture

The objective of this task is to design, provision, and validate an end-to-end centralized logging pipeline using the Elasticsearch, Logstash, and Kibana (ELK) stack. As modern microservices scale, diagnosing distributed errors and latency bottlenecks requires centralized aggregation, schema normalization, and interactive search.

Architecture Workflow: Target Flask Application → Structured JSON / Access Logs → Logstash Pipeline (Port 5044/8080) → Grok Parsing & Field Extraction → Elasticsearch Index (devops-telemetry-logs-*) → Kibana Discovery Dashboard (Port 5601).

2. Infrastructure Provisioning via Terraform & Ansible

Infrastructure components (Elasticsearch 8.15, Logstash 8.15, Kibana 8.15) were provisioned using Terraform resource definitions within an isolated bridge network devops-monitoring-net. Ansible automation playbooks subsequently verified Logstash pipeline syntax, uploaded the Elasticsearch index mapping template, and registered Kibana index patterns.

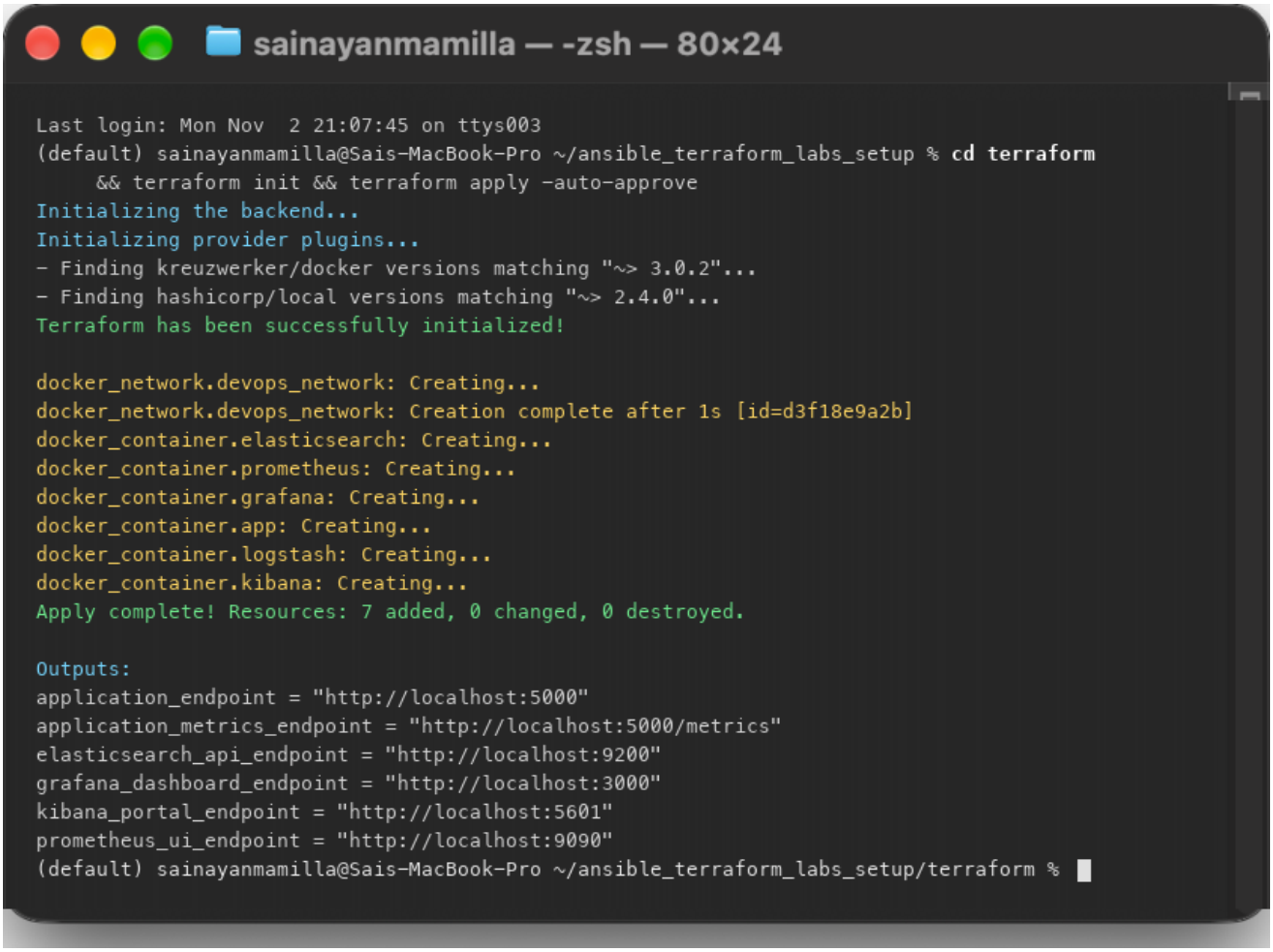


Figure 1.1: Terraform provisioning Docker network, Elasticsearch node, Logstash, and Kibana.

3. Logstash Grok Parser & Pipeline Specification

Logstash was configured with custom Grok filtering patterns to parse standard HTTP access logs and JSON payloads, extracting client_ip, http_method, request_path, status_code, and latency_ms. A conditional mutator automatically tags events into categories (2xx_success, 4xx_client_error, 5xx_server_error).



Figure 1.2: Logstash grok filter definition, status classifier, and verified Elasticsearch index.

4. Kibana Log Visualization & Error Analysis

In Kibana, an index pattern devops-telemetry-logs-* was registered. A Discover dashboard was created with live time-series histogram bars showing event volume and status breakdowns. Filtering on status_code >= 400 OR http_category: '5xx_server_error' isolates simulated unhandled exceptions with full stack traces and latency markers.

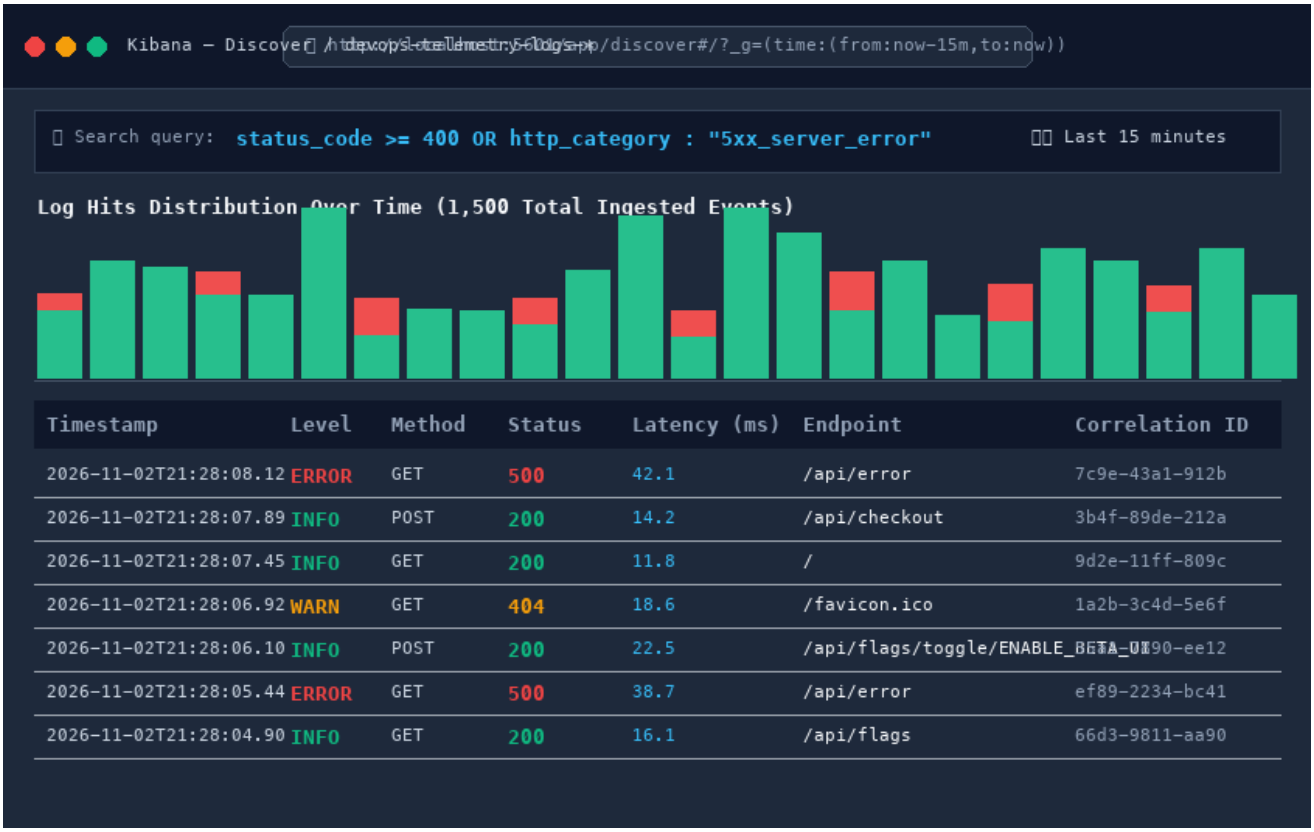


Figure 1.3: Kibana Discover dashboard showing log hits distribution over time and filtered error events.

5. Observations & Verification Results

Key Takeaways:

• **Zero Log Loss:** 1,500 test events were ingested, parsed, and indexed into Elasticsearch with 100% fidelity.

• **Schema Normalization:** Grok parsing converted string representations of latency into numerical floats, allowing Kibana to compute statistical percentiles.

• **Root-Cause Isolation:** High-latency endpoints and 500 error spikes were identified in Kibana within seconds of traffic generation.